Introduction to Image Processing with Python

Introduction to Image Processing with Python

Algorithms, Implementation and Practical Image Processing

Robert Elliott BSc (Hons), MBCS

First Edition – Draft

First edition – development draft.

Acknowledgements

This book grew from a desire to revisit and extend material I first encountered in 2001 while taking UQC146S1, *Introductory Image Processing in C*, at the University of the West of England (UWE). The module was taught by Ian Johnson, who was a lecturer at UWE at the time and is named as the module author in the surviving module specification[1]. His surviving lecture material provides the historical starting point for the practical image-processing work revisited in this book[2].

The historical material is used in this book to establish provenance and to recover the practical scope of the original module. The explanations, Python implementations, tests, notebooks and figures in this book are being developed as new material. Modern technical claims and algorithms will be supported by current specifications, primary sources, authoritative textbooks and official software documentation as appropriate.

Further acknowledgements will be added during development.

Preface

Image processing is an unusually effective way to learn numerical programming. An image is immediately recognisable, yet it can also be treated as structured numerical data. A small change to an algorithm can often be seen at once in the resulting image, while the same program raises deeper questions about data representation, numerical range, file formats, colour models, filtering, geometry and compression.

The starting point for this book is personal as well as historical. In 2001 I took UQC146S1, *Introductory Image Processing in C*, at the University of the West of England (UWE). The module was taught by Ian Johnson, a lecturer at UWE at the time and the module author named in the surviving specification [1]. The module combined a limited introduction to C with practical image-processing work[2]. Its course website directed students towards progressively implementing techniques introduced in lectures and maintaining a portfolio of image-processing functions [3]. The coursework specification explicitly described that portfolio as a library of functions and gave importance not only to code, but also to programming style, documentation and testing [4].

This book preserves that implementation-led spirit while moving the work into a modern Python environment. Algorithms are developed and understood before they are replaced by convenient library calls. NumPy, Pillow, OpenCV and scikit-image are valuable tools, but they become more useful once the behaviour beneath their APIs is understood.

A reusable Python package, `pyimage`, is developed alongside the text. The companion Jupyter notebooks provide executable chapter material, while a pytest suite verifies the library independently. The intention is that the code shown in the book should form part of a working software project rather than a set of isolated fragments.

This is a development edition. References, terminology, algorithms and implementation details will be checked chapter by chapter against appropriate primary and authoritative sources before the manuscript is considered ready for publication.

How to Use This Book

The book is designed to be read alongside its companion project. Each major technique is approached in four connected forms:

1. the chapter explains the concept, mathematics and algorithm;
2. the companion notebook develops and explores the implementation;
3. the finished implementation is added to the `pyimage` package; and
4. automated tests verify the behaviour of the reusable implementation.

The normal progression is therefore:

$$\textbf{theory} \rightarrow \textbf{algorithm} \rightarrow \textbf{notebook} \rightarrow \textbf{library} \rightarrow \textbf{tests} \rightarrow \textbf{library comparison}.$$

The library comparison comes last deliberately. Where a technique can be implemented directly, the book first constructs it from basic Python and/or NumPy operations. Established libraries are then used to compare behaviour, performance and API design.

Companion notebooks

The Jupyter notebooks mirror the chapter numbering. For example, `notebooks/17-edge-detection.ipynb` accompanies Chapter 17. Notebooks may include exploratory code and intermediate implementations that do not belong in the final package.

The `pyimage` package

The canonical reusable implementations live under `project/src/pyimage/`. Once an implementation has been developed and validated in a notebook, the polished function is added to the package and covered by the test suite.

Historical sign-off items

Some chapters contain a box labelled *Original portfolio/sign-off item*. The stars reproduce the relative weighting used in the historical material where that weighting can be verified. They are retained as a record of the original practical scope, not as assessment marks for this book.

Companion Project Files

The source distribution is organised so that the manuscript, executable notebooks and reusable Python package can evolve together.

```
introduction-to-image-processing-with-python/
|-- book/
|   |-- main.tex
|   |-- preamble.tex
|   |-- references.bib
|   |-- frontmatter/
|   |-- chapters/
|   |-- appendices/
|   `-- figures/
|-- notebooks/
|-- project/
|   |-- pyproject.toml
|   |-- src/pyimage/
|   |-- tests/
|   |-- examples/
|   `-- images/
`-- source-notes/
```

The notebooks are the executable companion to individual chapters. The `pyimage` package is the finished reusable implementation. The test suite is kept outside the notebooks so that the complete package can be verified from the command line or from the integrated terminal in Visual Studio Code.

Publication releases should eventually tag the companion project with a version corresponding to the edition of the book.

Conventions Used in This Book

Language

The text uses British English. Accordingly, terms such as *colour* and *greyscale* are preferred in prose. Library, API and file-format names are written exactly as their respective projects define them.

Code

Python source code is shown in monospaced listings. Longer examples may be loaded directly from the companion project so that the printed listing and the working source remain synchronised.

Shell commands

Commands intended for a terminal are shown without assuming a particular shell unless the command itself is platform-specific. Chapter 2 will establish the Anaconda, Visual Studio Code and Jupyter workflow used throughout the book.

Pixel coordinates

Unless otherwise stated, image arrays use row-major NumPy indexing. An element written as `image[y, x]` therefore identifies row `y` and column `x`. Where Cartesian coordinates are used in mathematics, the coordinate convention will be stated explicitly.

Historical weighting

Asterisks such as **** reproduce the historical relative weighting of portfolio/sign-off functions where a source is available. They do not indicate the importance of a topic in modern image processing.

References

Numbered citations are managed with BibLaTeX and Biber. Historical course material is cited for provenance. Technical descriptions should, wherever possible, cite the relevant specification, original paper, authoritative text or official software documentation.

Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CCD	Charge-Coupled Device
CMOS	Complementary Metal-Oxide-Semiconductor
DFT	Discrete Fourier Transform
FFT	Fast Fourier Transform
HSI	Hue, Saturation and Intensity
JPEG	Joint Photographic Experts Group
LUT	Lookup Table
PBM	Portable Bitmap
PGM	Portable Graymap
PNG	Portable Network Graphics
PNM	Portable Anymap / collective Netpbm family term
PPM	Portable Pixmap
RGB	Red, Green and Blue

Contents

List of Figures

List of Tables

Listings

Part I.

Foundations

1. Introduction to Image Processing

1.1. Learning Objectives

By the end of this chapter, you should be able to:

- explain what is meant by digital image processing;
- distinguish image processing from computer graphics;
- describe a digital image as numerical data;
- explain why arrays are central to image processing;
- describe the implementation-led approach used in this book; and
- identify the relationship between the book, its Jupyter notebooks and the `pyimage` package.

1.2. Introduction

Digital image processing is concerned with representing images as data and applying computational operations to that data. Depending on the application, those operations may enhance an image, transform its appearance, extract measurements, compare it with another image, reduce its storage requirements or prepare it for further analysis.

This book approaches image processing as both a mathematical subject and a programming discipline. The objective is not merely to learn which library function performs a task, but to understand enough of the underlying operation to implement, test and evaluate it.

Historical note

The origins of this book lie in UQC146S1, *Introductory Image Processing in C*, which the author took at the University of the West of England in 2001. The module was taught by Ian Johnson, a UWE lecturer at the time and the module author identified in the surviving specification[1]. The teaching material described the module as covering both the C programming language and basic image processing, with only a limited period spent introducing C before moving on to image-processing work [2]. This book keeps the same practical emphasis but uses Python as the implementation language.

1.3. Image Processing and Computer Graphics

The historical course material made a useful distinction between computer graphics and image processing: computer graphics is principally concerned with generating images, while image processing begins with existing image data and manipulates or analyses it. That distinction is useful for orientation, even though modern applications often combine the two areas.

In this book, an image is therefore primarily treated as data. Displaying the image is important, but the display is only one representation of the numeric values held by the program.

1.4. Images as Numerical Data

At its simplest, a greyscale digital image can be represented as a rectangular array of numbers. Each array element corresponds to a sample at a particular location in the image. In an 8-bit greyscale representation, a value of 0 is commonly associated with black and 255 with white, with intermediate values representing intermediate levels of brightness.

A tiny image can therefore be constructed directly with NumPy:

Listing 1.1: A small greyscale image represented as a NumPy array.

```
import numpy as np

image = np.array([
    [0,   0,  40,  40],
    [0,  60, 120,  40],
    [20, 100, 220, 80],
    [0,  40,  80,   0],
], dtype=np.uint8)
```

The choice of `uint8` is significant. It limits each array element to an unsigned 8-bit integer and therefore gives it a finite numeric range. Later chapters will examine the consequences of overflow, underflow, clipping and data-type conversion in detail.

1.5. Images as Arrays

The original course material emphasised that an image may be viewed logically as a two-dimensional array while still being stored in memory or on disk as a sequence of pixel values. That distinction remains important in modern Python: NumPy gives us a convenient multidimensional view, while the underlying data still has a concrete representation in memory.

For a greyscale image, a common array shape is:

$$H \times W, \tag{1.1}$$

where H is the image height and W is its width. A colour image may instead use a shape such as:

$$H \times W \times 3, \tag{1.2}$$

where the final dimension contains three colour components, commonly red, green and blue.

1.6. An Implementation-Led Approach

Many modern image-processing operations can be expressed with a single library call. That is valuable in production software, but it can conceal the decisions an algorithm makes about numeric range, border handling, interpolation, normalisation or colour representation.

The approach used throughout this book is therefore:

1. understand the operation conceptually;
2. express the operation mathematically where appropriate;
3. develop pseudocode;

4. implement the operation directly;
5. test it against known cases and boundary conditions;
6. examine the result visually and numerically;
7. move the validated implementation into the `pyimage` package; and
8. compare it with established library implementations.

This reflects an important feature of the historical portfolio. The coursework was not simply a request for output images: it required a library of functions, documentation and a test plan, and placed explicit value on programming style and demonstrated code[4].

1.7. The Companion Project

The practical work is divided between Jupyter notebooks and a conventional Python package. The notebooks are used to investigate algorithms, display intermediate results and experiment with parameter values. The package contains the reusable versions of the completed functions. A separate pytest suite provides repeatable verification.

Project files

Companion notebook
`notebooks/01-introduction.ipynb`

Reusable package
`project/src/pyimage/`

Automated tests
`project/tests/`

1.8. From C to Python

The historical module used C and deliberately exposed implementation details such as numeric types, pointers, manual allocation and linkable libraries. A Python implementation removes some of that machinery, but the underlying questions do not disappear. Images still have dimensions and data types; binary files still contain bytes in a defined order; numerical operations still have ranges and precision; reusable software still needs a clear API; and algorithms still need to be tested.

For that reason, the book will not treat Python as a way to avoid the technical content of the original course. Instead, Python allows more time to be spent on the image-processing algorithms while NumPy makes the representation of image data explicit and inspectable.

1.9. What the Book Will Build

Over the following chapters the `pyimage` package will grow to cover file I/O, colour conversion, point operations, convolution, filtering, edge detection, geometric operations, frame operations, compression and selected standard image formats. The historical portfolio and sign-off material provides part of the practical roadmap, while modern material extends the project beyond the scope of the original module.

1.10. Chapter Summary

A digital image can be treated as structured numerical data. Image processing operates on that data to transform, enhance, combine, compress or analyse an image. This book develops those operations by implementing them before relying on high-level library equivalents. The chapter notebooks support exploration; the `pyimage` package holds the finished implementations; and the automated tests provide independent verification.

1.11. Review Questions

1. What is the practical distinction between image processing and computer graphics used in this book?
2. Why is an array a useful representation of a digital image?
3. What information is expressed by the shape $H \times W \times 3$?
4. Why might a direct implementation be useful even when a library already supplies the same operation?
5. What different roles do the notebooks, the `pyimage` package and the test suite play in the companion project?

References

[1] University of the West of England, *UQC146S1: Introductory image processing in C – module specification*, Module specification, University of the West of England, Version 1; valid from September 2000; definitive version dated 1 September 2001; module author: Ian Johnson, 2001.

[2] I. Johnson, *UQC146S1: Introductory image processing in C*, Lecture slides, University of the West of England, Surviving course teaching material; introductory slide set.

[3] I. Johnson. 'UQC146S1: Introduction to image processing in C.' Historical course webpage; surviving local copy used for research, University of the West of England. [Online]. Available: `http://www.cems.uwe.ac.uk/~irjohnso/uqc146s1.html`.

[4] I. Johnson, *UQC146S1 – an introduction to image processing in C: Portfolio of work*, Coursework specification, University of the West of England, The surviving document gives a submission date of 18 April 2002, Sep. 2001.